

Champion Document - Feature 5

Name: _____ Feature: Meta-Progression / Point Economy TL: TL4

Scope statement. This champion document defines the point economy that survives individual runs. It follows the sample document's scenario, data-flow, and scheduling structure while making every public boundary independently testable.

1. Introduction

The Meta-Progression / Point Economy feature tracks points earned across completed runs and lets the player exchange those points for permanent unlocks. One persistent **EconomyManager** is the authoritative owner of the balance and unlock set. All mutations are represented as **CurrencyTransaction** objects so payouts, purchases, and refunds each have explicit, independently testable balance rules.

At the team boundary, Feature 5 receives **PurchaseRequest**, **RefundRequest**, and **RunEndPoints** from the Dungeon Crawler Complete Team System. Internally, **RunEndPoints** invokes **CreditRunEnd(pointsEarned)**. **PurchaseRequest** invokes **TryPurchase(UnlockID)**, which verifies that the ID exists, is not already owned, and is affordable before a **PurchaseFeatureTransaction** debits its exact configured cost and marks it unlocked. Feature 5 returns **Balance**, **PurchaseResult**, and **UnlockState** to the same integration boundary.

What this feature does not do

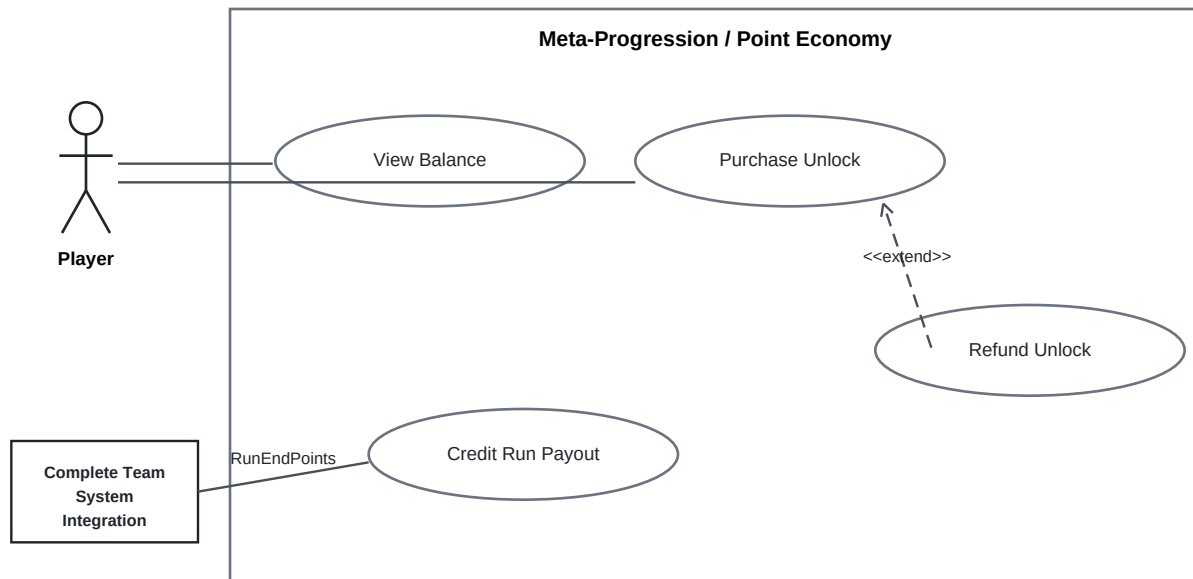
It does **not** calculate run performance or decide how many points a run deserves; the complete-system integration supplies the final non-negative **RunEndPoints** value originating from Feature 1. It does **not** render shop screens, buttons, or balance labels. It does **not** implement unlocked gameplay content; the team system distributes **UnlockState** to consumers such as Features 4 and 6. It does **not** permit direct writes to the balance: the private balance field is changed only by **EconomyManager** while executing a transaction.

Design responsibilities

GOF: Singleton provides one persistent **EconomyManager** across scene/run changes. **Private Class Data:** **EconomyManager** exclusively owns an extracted **EconomyData** object containing the private balance and unlock set; callers receive only queries and typed results. **GRASP Controller:** **EconomyManager** receives all economy requests. **Protected Variations:** callers depend on the **CurrencyTransaction** contract, so transaction subclasses can change without changing call sites.

Polymorphism checkpoint: **CurrencyTransaction** is an abstract, non-MonoBehaviour C# class, but **Execute(EconomyManager mgr)** is a concrete virtual method with default flat-credit behavior. Each derived type overrides it. A **CurrencyTransaction** reference dispatches to the runtime subclass. Removing virtual/override makes every call use the flat base payout; a purchase visibly credits rather than debits points. **MonoBehaviour** is not responsible for this binding.

2. Use Case - Purchase a Permanent Unlock



Use-case diagram. Player intent and run-end data enter Feature 5 through the complete-team integration boundary.

Scenario MP1 - Purchase Unlock

Summary: The player spends persistent points on one permanent unlock.

Primary actor: Player **Supporting actors:** Complete Team System Integration, Shop UI, Save System

Preconditions: Economy state is loaded; the requested UnlockID exists in the catalog.

Priority: 1 (must have) **ID:** MP1

Basic sequence

Step 1: The player selects an unlock in the shop UI.

Step 2: Process 0 sends **PurchaseRequest(id: UnlockID)** to Feature 5; the integration adapter calls TryPurchase(id).

Step 3: EconomyManager validates that the ID exists, is not already unlocked, and the current balance is at least the configured cost.

Step 4: EconomyManager creates PurchaseFeatureTransaction(id, cost) and calls its overridden Execute method.

Step 5: The transaction debits exactly *cost* points and adds *id* to the persistent unlock set.

Step 6: EconomyManager serializes the balance and unlock set.

Step 7: Feature 5 returns **Balance**, **PurchaseResult**, and **UnlockState** to Process 0, which distributes the result to the UI and content systems.

Exceptions

Step 3a: The ID is unknown. No state changes, no events fire, and TryPurchase returns false.

Step 3b: The unlock is already owned. No debit occurs, no events fire, and TryPurchase returns false.

Step 3c: Balance is less than cost. No debit occurs, no unlock is added, and TryPurchase returns false.

Step 6a: Persistence fails. The in-memory mutation is rolled back to the pre-transaction snapshot, a persistence-error event is raised, and TryPurchase returns false.

Postconditions on success: balanceAfter = balanceBefore - configuredCost(id); id is present in the saved unlock set; exactly one balance event and one unlock event were emitted.

Postconditions on failure: balance and unlock set equal their pre-call values; return value is false.

2. Use Case - Run Payout and Refund (continued)

Scenario MP2 - Credit Run-End Points

Primary actor: Complete Team System Integration **Originating subsystem:** Feature 1 **ID:** MP2

Preconditions: A run has reached its terminal state; EconomyManager has loaded.

Basic sequence

- Step 1: The complete-system integration sends **RunEndPoints** to Feature 5.
- Step 2: The integration adapter calls **CreditRunEnd(pointsEarned: int)**.
- Step 3: EconomyManager verifies pointsEarned is non-negative.
- Step 4: EconomyManager creates RunEndPayoutTransaction(pointsEarned) and calls Execute.
- Step 5: The transaction sets balanceAfter = balanceBefore + pointsEarned.
- Step 6: EconomyManager persists state and returns the updated Balance to the team integration boundary.

Exceptions

Step 3a: pointsEarned is negative. The call is rejected with ArgumentOutOfRangeException; state is unchanged.

Step 6a: Persistence fails. The credit is rolled back, PersistenceFailed(RUN_END) fires, and the previously persisted Balance remains authoritative.

Postconditions: On success, the exact payout is visible through GetBalance after reload and the updated Balance is returned to Process 0.

Scenario MP3 - Refund an Unlock

Primary actor: Player **ID:** MP3

Preconditions: The selected unlock is owned and marked refundable; EconomyManager has loaded.

Basic sequence

- Step 1: The player selects Refund for an owned unlock.
- Step 2: Process 0 sends **RefundRequest(id: UnlockID)** to TeamIntegrationAdapter.
- Step 3: The adapter creates RefundTransaction; EconomyManager validates ownership, refund eligibility, and the configured refund amount.
- Step 4: RefundTransaction removes the unlock and credits exactly the configured refund amount.
- Step 5: EconomyManager persists state and returns Balance, PurchaseResult, and UnlockState to Process 0.

Exceptions

Step 3a: The unlock is not owned. No state or balance change occurs.

Step 3b: The unlock is non-refundable. The request is rejected with an explicit NON_REFUNDABLE result.

Step 5a: Persistence fails. Both the balance credit and unlock removal are rolled back.

Acceptance checks derived from the scenarios

For each branch, tests assert the exact return/result code, exact balance delta, exact unlock-set membership, event count, and serialized state after reload. This makes every success and exception branch observable without relying on UI behavior.

3. Interface Contract

All integer amounts are whole points. The invariant is **balance** ≥ 0 . UnlockID is a strongly typed enum/value object whose serialized key is stable across builds. Public callers never receive a mutable reference to internal state.

Public inputs and outputs

Member	Typed input	Typed output	Testable contract
GetBalance()	None	int	Returns current persisted balance; value is ≥ 0 ; no mutation or event.
TryPurchase(id)	UnlockID id; must resolve in catalog	bool	True iff one debit and one unlock occur. False leaves all state unchanged.
CreditRunEnd(pointsEarned)	int pointsEarned, ≥ 0	void	Adds exactly pointsEarned for each valid RunEndPoints input; negative input throws.

Integration-only refund route: RefundRequest is handled by TeamIntegrationAdapter, which creates RefundTransaction. It is not an additional member of the specified three-method public interface.

Published events

Event	Payload	Emission rule / assertion
BalanceChanged	{ oldBalance:int, newBalance:int, reason:TransactionType }	Exactly once after successful persistence; newBalance-oldBalance equals transaction delta.
UnlockStateChanged	{ id:UnlockID, isUnlocked:bool }	Exactly once after successful purchase/refund persistence; payload matches saved set.
PersistenceFailed	{ type:TransactionType, errorCode:SaveError }	Once on save failure, after rollback; GetBalance equals its pre-call value.

Transaction test matrix

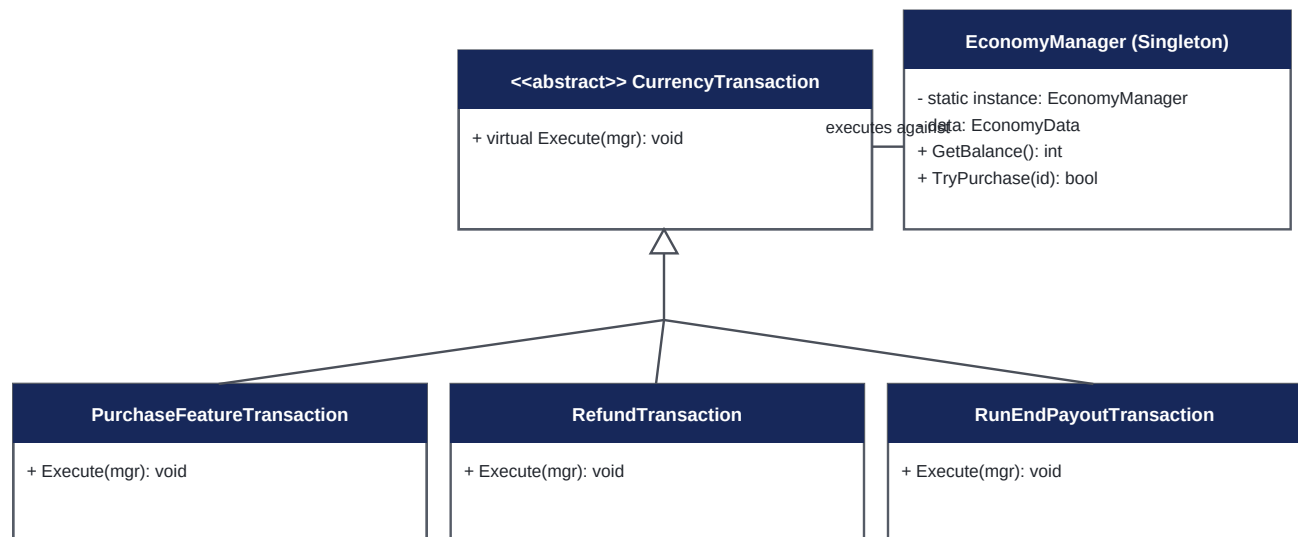
Purchase: start 100, cost 35 -> end 65 and unlocked=true. Insufficient case start 34 -> end 34, false, no event.

Refund: start 65, refund 20, owned=true -> end 85 and owned=false.

Run payout: start 65, earned 40 -> end 105; a second valid call with 40 -> end 145. Duplicate-call prevention belongs to the originating run/game-state integration because CreditRunEnd carries no run identifier.

Round trip: serialize balance 105 plus unlock set, construct a fresh manager, deserialize, then assert balance/unlock equivalence.

3. Interface Contract - Internal Structure (continued)



Class relationship view. The virtual Execute operation is the protected variation point.

Internal classes and invariants

EconomyManager (Singleton): the only MonoBehaviour in this feature; one instance survives scene/run transitions and duplicate instances destroy themselves. It exclusively owns EconomyData and an injected persistence adapter. It does not expose mutable economy fields.

EconomyData (Private Class Data): ordinary internal C# class containing private balance and unlock-set attributes. EconomyManager constructs and owns it; callers can query state but cannot mutate it directly.

CurrencyTransaction: abstract, non-MonoBehaviour C# class with concrete **virtual void Execute(EconomyManager mgr)** implementing the flat default credit. Derived purchase, refund, and payout classes use **override** for their intended mutation logic.

PurchaseFeatureTransaction: delta = -cost and adds one UnlockID. **RefundTransaction:** delta = +refundAmount and removes one UnlockID. **RunEndPayoutTransaction:** delta = +pointsEarned.

Persistence record

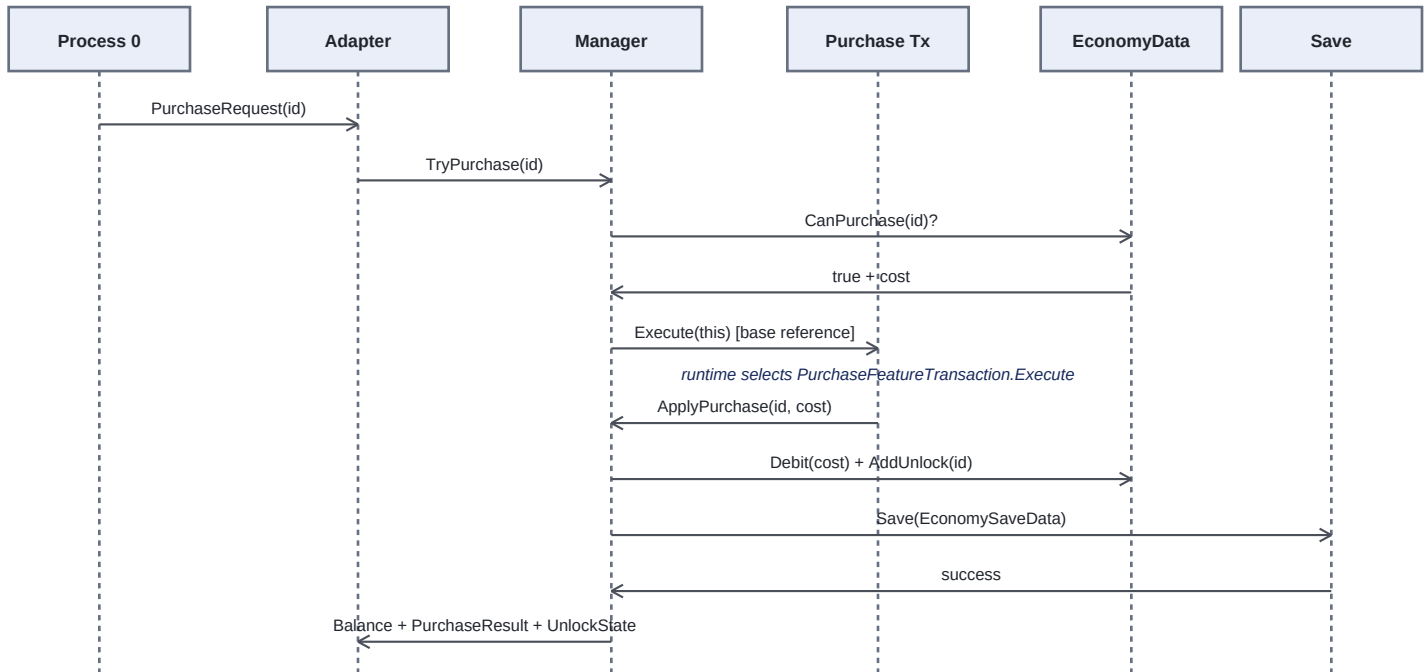
EconomySaveData = { schemaVersion:int, balance:int, unlockedIds:List<string> }. Deserialization rejects negative balance, unknown schema versions, and duplicate unlock IDs. Migration code may translate older schema versions, but it may never silently reset a valid balance.

Dependency boundary

All cross-feature traffic is routed through the Dungeon Crawler Complete Team System. Feature 5 accepts PurchaseRequest, RefundRequest, and RunEndPoints; it returns Balance, PurchaseResult, and UnlockState. Feature 1 originates RunEndPoints, while team consumers such as Features 4 and 6 receive UnlockState through Process 0. The save adapter remains an internal dependency and is injected for deterministic persistence tests.

3. Interface Contract - Purchase Sequence (continued)

This interaction shows the successful purchase path and includes the dynamic-binding call site. TeamIntegrationAdapter translates the team message into the specified public method; EconomyManager acts as controller and command invoker.



Purchase sequence. The call is made through a `CurrencyTransaction` reference, but the runtime object is `PurchaseFeatureTransaction`.

Failure branches

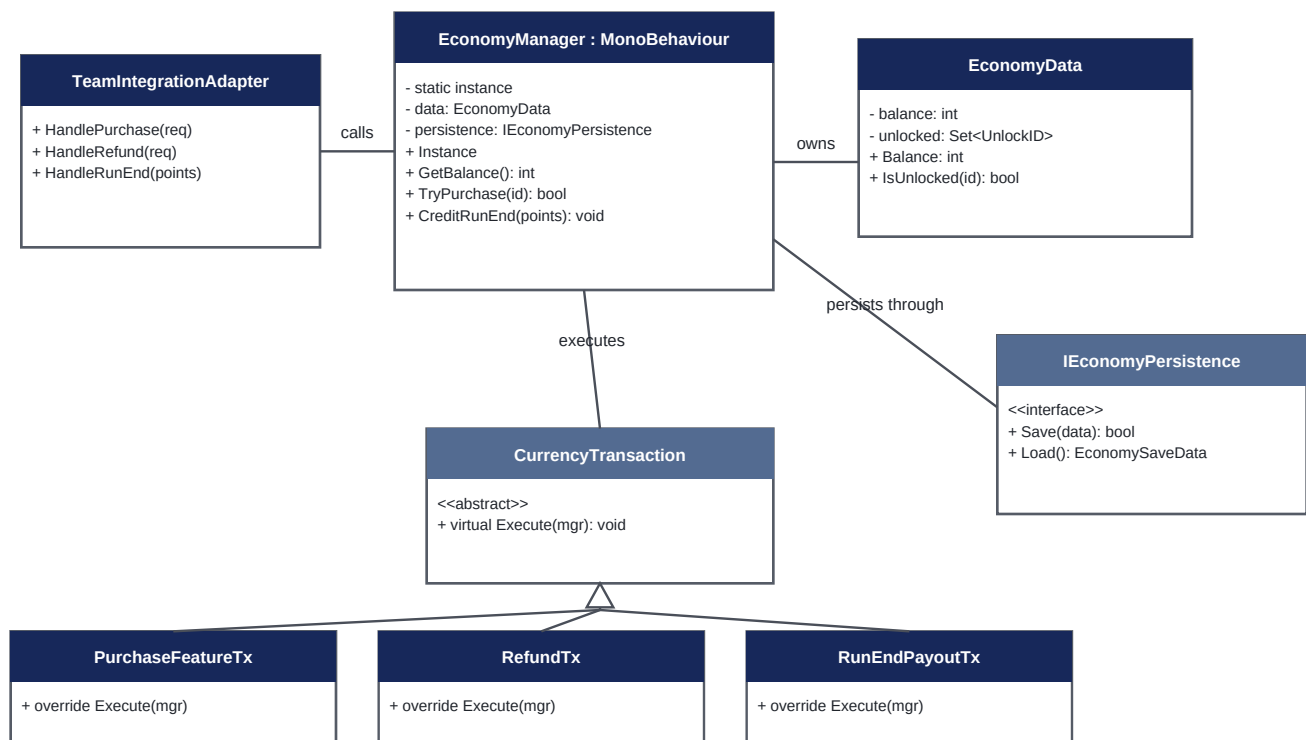
Validation failure: `EconomyData` returns unknown, already-owned, or insufficient-balance status; no transaction executes and Process 0 receives a failed `PurchaseResult` with unchanged `Balance` and `UnlockState`.

Persistence failure: `EconomyManager` restores the pre-transaction `EconomyData` snapshot and returns failure. No success event is emitted.

Dynamic-binding assertion

Given a `CurrencyTransaction` reference whose runtime object is `PurchaseFeatureTransaction`, calling `Execute` must produce $\text{delta} = -\text{cost}$. The same call with `RefundTransaction` must produce $+\text{refundAmount}$, and with `RunEndPayoutTransaction` must produce $+\text{pointsEarned}$. If `virtual/override` is removed, all three calls execute the base flat credit and the purchase test fails visibly.

3. Interface Contract - Complete Class Diagram (continued)



Complete structural view: Singleton, extracted Private Class Data, persistence boundary, integration adapter, and transaction inheritance.

Pattern-role mapping

Pattern / principle	Role	Feature class
Singleton	Single instance and global accessor	EconomyManager
Private Class Data	Extracted hidden state object	EconomyData
Command-like transaction hierarchy	Base command / concrete commands	CurrencyTransaction and three subclasses
GRASP Controller	Coordinates requests and persistence	EconomyManager
Protected Variations	Stable virtual transaction boundary	Execute(EconomyManager)

Unity boundary: EconomyManager alone inherits MonoBehaviour for Awake(), duplicate-instance destruction, and DontDestroyOnLoad. EconomyData, TeamIntegrationAdapter, persistence implementations, and all CurrencyTransaction classes are ordinary C# objects.

4. Data Flow Diagrams

The context diagram treats the whole economy feature as one process. Every incoming flow is transformed into an outgoing result, event, or persisted record. The Save System store is both written and read, preventing a black hole; requests never produce data without an input or stored source, preventing miracles.

Diagram 0 - System Context



Gane-Sarson notation: squared entities, numbered process, open-ended-style data store, and labeled data flows.

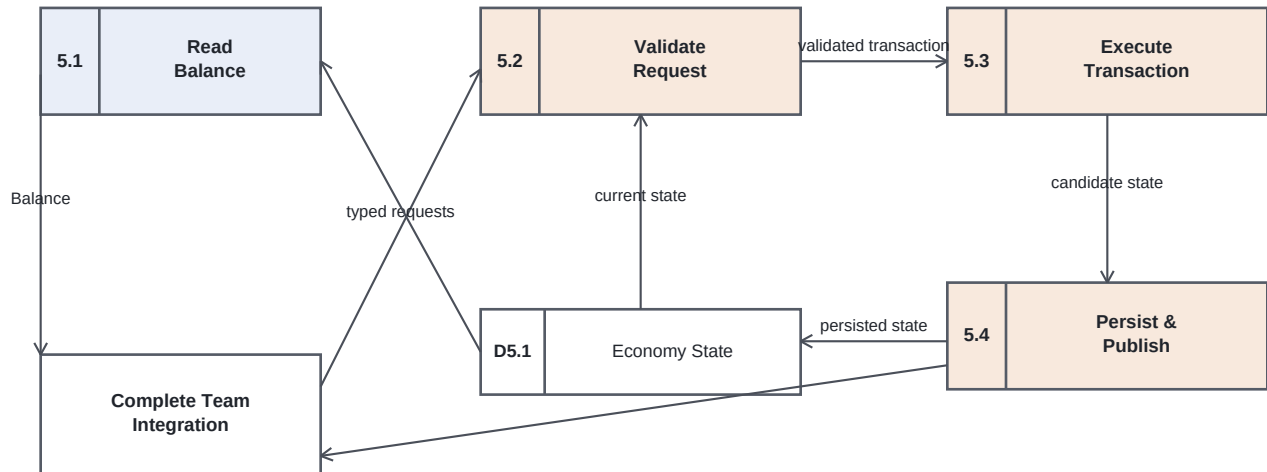
Flow balancing

Entity / data store	Inbound to Process 5	Outbound from Process 5
Complete Team System	PurchaseRequest; RefundRequest; RunEndPoints	Balance; PurchaseResult; UnlockState
Save System	saved balance and unlock IDs	serialized economy state

4. Data Flow Diagrams - Feature Zoom (continued)

Process 5 is decomposed below. The boundary flows balance with Diagram 0: PurchaseRequest, RefundRequest, and RunEndPoints enter; Balance, PurchaseResult, and UnlockState leave; saved economy state crosses only the persistence boundary.

Diagram 5 - Meta-Progression / Point Economy



D5.1 is the feature-owned logical economy-state store; the Save System is its physical persistence adapter at the context boundary.

Primitive process descriptions

5.1 Read Balance. Reads balance from D5.1 and supplies the current Balance output. It cannot write state.

5.2 Validate Request. Combines PurchaseRequest, RefundRequest, or RunEndPoints with current balance/unlock data. It emits either a validated transaction or an explicit rejection PurchaseResult back to Process 0. No mutation occurs here.

5.3 Execute Transaction. Invokes Execute through a CurrencyTransaction reference; runtime dispatch selects the concrete override. It outputs candidate state plus a typed result and cannot publish success before persistence.

5.4 Persist & Publish. Writes candidate state to D5.1. On success it returns Balance, PurchaseResult, and UnlockState to Complete Team Integration; on failure it restores the snapshot and returns a failure result.

No black holes or miracles check

No black holes: purchase/refund/payout requests always end in a typed success or rejection result; a candidate mutation always ends in persisted state or rollback/error. **No miracles:** balance output comes from D5.1; debits require a catalog cost and purchase request; credits require a run payout or eligible refund; unlock events require a successful persisted mutation.

5. Timeline

Estimates are implementation hours, not elapsed calendar time. They total **52 hours**: Design 9, Coding 27, Testing 11, Documentation 5. TL4 progress should be tracked against each row; overruns should be recorded rather than hidden by changing completed-hour estimates.

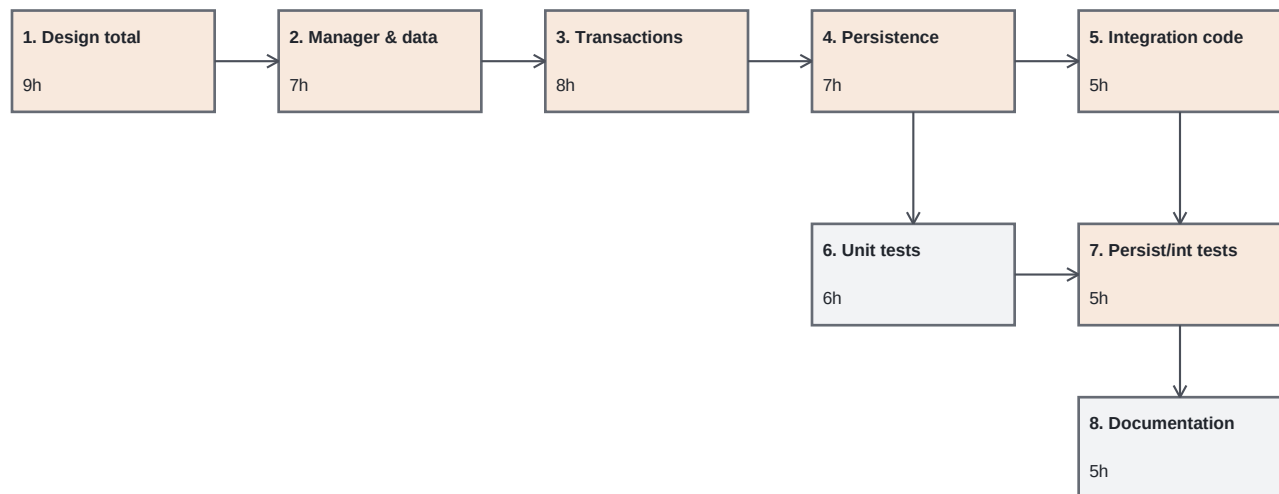
Work item	Category	Hours	Predecessor(s)
1. Define economy rules and invariants	Design	2	-
2. Define unlock costs/refund rules	Design	1	1
3. Design transaction hierarchy	Design	2	1
4. Design Singleton lifecycle	Design	1	1
5. Design EconomyData extraction	Design	1	1
6. Specify three-method public API	Design	1	2-5
7. Define events and save schema	Design	1	6
8. Implement UnlockID/result types	Coding	1	6
9. Implement EconomyData	Coding	3	5,7
10. Implement Singleton lifecycle	Coding	3	4
11. Implement EconomyManager API	Coding	3	8-10
12. Implement CurrencyTransaction base	Coding	2	3
13. Implement purchase transaction	Coding	2	9,12
14. Implement refund transaction	Coding	2	9,12
15. Implement payout transaction	Coding	2	9,12
16. Implement validation/rollback	Coding	3	13-15

5. Timeline - Fine-Grained Tasks (continued)

Work item	Category	Hours	Predecessor(s)
17. Implement serialization DTO	Coding	2	7,9
18. Implement save/load adapter	Coding	3	17
19. Connect Process 0 RunEndPoints	Coding	1	11,15
20. Connect purchase/refund messages	Coding	1	11,13,14
21. Connect Balance/Result/UnlockState	Coding	1	11
22. Test Singleton duplicates	Testing	1	10
23. Test purchase branches	Testing	2	13,16
24. Test refund branches	Testing	2	14,16
25. Test payout and virtual dispatch	Testing	2	12,15
26. Test serialization round trip	Testing	2	17,18
27. Test persistence rollback	Testing	1	16,18
28. Test Process 0 integration	Testing	1	19-21
29. Write API comments	Documentation	1	11-21
30. Finalize UML/DFD diagrams	Documentation	2	19-21
31. Record test evidence/limits	Documentation	1	22-28
32. Final champion review	Documentation	1	29-31

Category check: Design 9 + Coding 27 + Testing 11 + Documentation 5 = **52 hours**. Process 0 integration is correctly classified as coding; only its verification is classified as testing.

PERT dependency view



Highlighted chain: model -> interfaces -> manager -> transactions -> persistence/integration testing -> documentation. Parallel test work begins as soon as its implementation predecessor is stable.

Estimate accountability

At each checkpoint, record actual hours beside the estimate and note scope changes. Completion means: all three transaction types produce exact deltas in unit tests; serialize/deserialize round-trip preserves balance and unlocks; forced persistence failure rolls back; Process 0 can submit RunEndPoints and purchase/refund requests; Process 0 receives exact Balance, PurchaseResult, and UnlockState outputs; API and champion documentation are current.